Ex.no: 12b)
Date:

Chat client server using TCP/UDP socket

AIM:

To write and execute a c/java socket program to implement a two way chat client and chat server and the server correctly echoed the received message back to the client.

Algorithm:

1) Create a socket bind it to a designed port.
2) Listen and accept connection from the client.
3) Start the communication loop.
4) Close the socket when chat ends.

client side.

1) Create a socket and connect to the server IP and port.
2) Start a communication loop.
3) Close the socket connection.

TCP server

```
import socket
S= socket.socket()
S = bind (('local host', 5000))
S.listen (1)
C, addr = S.accept ()
While True:
    msg = C.recv(1024).decode()
    if msg == "bye";
```

```
            break
        print ("client : ", msg)
        reply = input ("server")
        c.send (reply.encode())
    c.close()
    s.close()
```

Top client

```
    import socket
s = socket.socket()
s connect (c.localhost: 5000))
while True:
        msg = input ("client : ")
        s.send (msg.encode())
        break
        reply = s.recv (1024).decode()
    s.close()
```

```
import socket
S = socket.socket (socket AF-INET, socket.
    sock.DGRAM)
S = bind (('localhost', 5000)).

While True
        msg.add = s.recvfrom (1024)
        msg = msg = decode()
        print ("client : ", msg)
        if msg = "bye"
            break
```

output

TCP chat server

TCP chat server started

client connected

Client: Hello

server: Hi°

Client: bye

chat ended

Tcp chat client

client: Hello

Server: Hi°

Client: bye

Chat ended

UDP chat server

UDP chat server started

waiting for message.

client: Hello

Server: Hi°

Client: bye

chat ended

UDP chat client

Client: Hello

Server: Hi°

Client: bye

Chat ended

```
s.close()
```

```
import socket
s=socket.socket(socket.AF_INET, saket.DGRAR)
while True :
    msg : input ("client : ")
    s.sendto (msg.encode(),(('localhost': 5000))
    if msg == "bye":
        break
    s.close()
```

Result :-

chat communication between the client and server was successfully implemented using TCP and UDP socket in python.